

SQL

Manuale di preparazione e ripasso di inizio anno

A. Adinolfi

Dipartimento di Informatica

Liceo Scientifico Statale “Nino Cortese” – Maddaloni (CE)

Abstract—Questo manuale raccoglie, in forma sintetica, le nozioni fondamentali di basi di dati relazionali e del linguaggio SQL necessarie per affrontare la scheda di ripetizione di inizio anno sul database GESTIONE_SCUOLA. Sono richiamati i concetti di tabella, chiave primaria ed esterna, le clausole SELECT, WHERE, ORDER BY, le funzioni di aggregazione, GROUP BY, HAVING, le operazioni di JOIN e le sottoquery. Il documento non sostituisce il libro di testo, ma ne costituisce un compendio operativo da consultare prima e durante lo svolgimento degli esercizi.

Index Terms—SQL, basi di dati relazionali, SELECT, JOIN, GROUP BY, sottoquery

I. INTRODUZIONE AI DATABASE RELAZIONALI

Un database relazionale organizza i dati in *tabelle* (o relazioni), ciascuna composta da *record* (le righe) e *attributi* (le colonne). Ogni tabella descrive un insieme omogeneo di informazioni: per esempio, nel database GESTIONE_SCUOLA la tabella STUDENTI contiene un record per ciascuno studente, con attributi come nome, cognome e classe.

Le tabelle non sono isolate: sono collegate fra loro tramite *relazioni*, realizzate per mezzo delle chiavi. Comprendere queste relazioni è il prerequisito indispensabile per scrivere query corrette, in particolare quando sono coinvolte più tabelle contemporaneamente.

II. CHIAVI PRIMARIE ED ESTERNE

La *chiave primaria* (*primary key*) è l'attributo, o l'insieme di attributi, che identifica in modo univoco ogni record di una tabella: non possono esistere due record con lo stesso valore di chiave primaria. La *chiave esterna* (*foreign key*) è un attributo che fa riferimento alla chiave primaria di un'altra tabella, e realizza così il collegamento fra le due.

```
1 CREATE TABLE VOTI (  
2     id_voto      INT PRIMARY KEY,  
3     id_studente INT REFERENCES STUDENTI(id_studente),  
4     id_materia  INT REFERENCES MATERIE(id_materia),  
5     voto        INT,  
6     data_voto   DATE  
7 );
```

Errore tipico: confondere le due chiavi. La chiave primaria vive “nella propria” tabella e la identifica; la chiave esterna vive in una tabella e punta alla chiave primaria di un'altra.

III. L'ISTRUZIONE SELECT E LA CLAUSOLA WHERE

L'istruzione SELECT restituisce un insieme di righe estratte da una o più tabelle. La clausola FROM indica la tabella di origine, la clausola WHERE filtra le righe in base a una condizione.

```
1 SELECT nome, cognome  
2 FROM STUDENTI  
3 WHERE classe = '4A';
```

L'asterisco SELECT * restituisce tutte le colonne della tabella; è comodo per esplorare i dati, ma è preferibile elencare esplicitamente le colonne necessarie in una query definitiva.

TABLE I
OPERATORI DI CONFRONTO

Operatore	Significato
=	uguale
<> oppure !=	diverso
< > <= >=	minore, maggiore, minore/maggiore o uguale

IV. OPERATORI LOGICI E CONDIZIONI COMPOSTE

Le condizioni della clausola WHERE si combinano con gli operatori logici AND, OR e NOT. Alcuni operatori specifici rendono più leggibili condizioni ricorrenti:

TABLE II
OPERATORI DI FILTRO SPECIFICI DI SQL

Operatore	Uso
BETWEEN a AND b	valore compreso fra a e b (estremi inclusi)
IN (v1, v2, ...)	valore appartenente a un insieme
LIKE 'M%'	corrispondenza a un modello di testo
IS NULL	valore mancante (assenza di dato)

```
1 SELECT *  
2 FROM STUDENTI  
3 WHERE classe IN ('4A', '4B')  
4 AND cognome LIKE 'M%';
```

Nel modello LIKE, il simbolo % rappresenta una sequenza qualunque di caratteri (anche vuota), mentre _ rappresenta esattamente un carattere.

Errore tipico: un valore NULL non si confronta mai con = NULL, ma esclusivamente con IS NULL oppure IS NOT NULL.

V. ORDINAMENTO E DUPLICATI: ORDER BY E DISTINCT

La clausola ORDER BY ordina le righe del risultato secondo una o più colonne, in ordine crescente (ASC, opzione predefinita) o decrescente (DESC). La parola chiave DISTINCT elimina le righe duplicate dal risultato.

```

1 SELECT DISTINCT classe
2 FROM STUDENTI
3 ORDER BY classe ASC;

```

VI. FUNZIONI DI AGGREGAZIONE

Le funzioni di aggregazione calcolano un valore di sintesi a partire da un insieme di righe, restituendo un unico risultato per l'intera tabella (o per ciascun gruppo, se combinate con GROUP BY, si veda la sezione successiva).

TABLE III
FUNZIONI DI AGGREGAZIONE

Funzione	Calcola
COUNT(*)	numero di righe (inclusi i NULL)
COUNT(col)	numero di valori non NULL in col
SUM(col)	somma dei valori
AVG(col)	media dei valori
MIN(col) / MAX(col)	valore minimo / massimo

```

1 SELECT COUNT(*), AVG(voto), MAX(voto), MIN(voto)
2 FROM VOTI;

```

Errore tipico: confondere COUNT(*), che conta tutte le righe, con COUNT(colonna), che ignora i valori NULL presenti in quella colonna.

VII. RAGGRUPPAMENTO: GROUP BY E HAVING

La clausola GROUP BY suddivide le righe in gruppi in base al valore di una o più colonne, permettendo di applicare una funzione di aggregazione separatamente a ciascun gruppo.

```

1 SELECT id_materia, AVG(voto)
2 FROM VOTI
3 GROUP BY id_materia;

```

Regola fondamentale: nella lista del SELECT possono comparire soltanto le colonne presenti nel GROUP BY oppure il risultato di una funzione di aggregazione; qualunque altra colonna genera un errore o un risultato ambiguo.

La clausola HAVING filtra i gruppi già formati, in base a una condizione che coinvolge tipicamente una funzione di aggregazione; si distingue da WHERE, che invece filtra le singole righe prima del raggruppamento.

```

1 SELECT id_materia, AVG(voto) AS media
2 FROM VOTI
3 GROUP BY id_materia
4 HAVING AVG(voto) > 7;

```

TABLE IV
WHERE CONTRO HAVING

WHERE	HAVING
filtra le singole righe	filtra i gruppi
agisce prima del GROUP BY	agisce dopo il GROUP BY
non ammette funzioni di aggregazione	ammette funzioni di aggregazione

VIII. INTERROGAZIONI SU PIÙ TABELLE: JOIN

Quando i dati necessari sono distribuiti su più tabelle collegate da chiavi, occorre una operazione di JOIN, che combina le righe di due tabelle sulla base di una condizione di corrispondenza (clausola ON), tipicamente fra chiave esterna e chiave primaria.

```

1 SELECT S.nome, S.cognome, V.voto
2 FROM STUDENTI AS S
3 INNER JOIN VOTI AS V
4 ON S.id_studente = V.id_studente;

```

L'INNER JOIN restituisce soltanto le righe per cui esiste corrispondenza in entrambe le tabelle: uno studente privo di voti non comparirebbe nel risultato precedente. Il LEFT JOIN restituisce invece tutte le righe della tabella di sinistra, anche quando non esiste corrispondenza nella tabella di destra (i campi corrispondenti risultano NULL).

```

1 SELECT S.nome, S.cognome, V.voto
2 FROM STUDENTI AS S
3 LEFT JOIN VOTI AS V
4 ON S.id_studente = V.id_studente
5 WHERE V.id_voto IS NULL;

```

Quest'ultima query, in particolare, individua gli studenti che non hanno ancora ricevuto alcun voto: uno schema molto utile ogniquale si debbano cercare le "assenze di corrispondenza" fra due tabelle.

Errore tipico: dimenticare la condizione ON produce un prodotto cartesiano, cioè ogni riga della prima tabella combinata con ogni riga della seconda, generando risultati privi di senso e spesso di dimensioni molto elevate.

IX. SOTTOQUERY

Una sottoquery è una query annidata all'interno di un'altra, usata quando il valore di confronto non è noto a priori ma deve essere calcolato dinamicamente (per esempio una media generale).

```

1 SELECT nome, cognome
2 FROM STUDENTI S
3 WHERE (
4     SELECT AVG(voto) FROM VOTI V
5     WHERE V.id_studente = S.id_studente
6 ) > (
7     SELECT AVG(voto) FROM VOTI
8 );

```

Una sottoquery può comparire nella clausola WHERE (come nell'esempio), oppure nella clausola FROM, dove il suo risultato viene trattato come se fosse una tabella temporanea.

Attenzione: una sottoquery non è sempre necessaria; se il problema si risolve con un semplice JOIN o con GROUP BY/HAVING, preferire la soluzione più semplice rende la query più leggibile ed efficiente.

X. ISTRUZIONI DI MODIFICA DEI DATI

Oltre all'interrogazione dei dati (SELECT), SQL include istruzioni per modificarli: INSERT aggiunge un nuovo record, UPDATE modifica record esistenti, DELETE li rimuove. In tutti e tre i casi, quando si opera su un sottoinsieme di righe, la clausola WHERE è essenziale: se omessa in UPDATE o DELETE, l'istruzione si applica a tutte le righe della tabella.

```
1 INSERT INTO STUDENTI (id_studente, nome, cognome, classe)
2 VALUES (9, 'Anna', 'Bruni', '4A');
3
4 UPDATE STUDENTI
5 SET classe = '4B'
6 WHERE id_studente = 9;
7
8 DELETE FROM STUDENTI
9 WHERE id_studente = 9;
```

XI. SINTESI E CONSIGLI PER LO SVOLGIMENTO

Di fronte a un problema, conviene procedere per gradi: (i) individuare quali tabelle contengono i dati necessari e quali chiavi le collegano; (ii) stabilire se serve un JOIN e, in caso affermativo, se debba essere INNER o LEFT; (iii) valutare se il risultato richiede un raggruppamento (GROUP BY) e un eventuale filtro sui gruppi (HAVING); (iv) chiedersi se il confronto richiesto dipende da un valore che va prima calcolato con una sottoquery. Scrivere la query un pezzo alla volta, verificandone il risultato a ogni aggiunta di clausola, riduce significativamente gli errori rispetto a scrivere l'istruzione completa in un solo passaggio.